High-Level Architecture & Component Guide

JIRA MEAL-61 • Version 1.3 • 01 November 2026

Practical system map for developing and integrating the Meal Connect MVP. This guide follows the MEAL-5 v1.3 architecture baseline.

1. Purpose and scope

Meal Connect is a browser-based surplus-food coordination MVP. Restaurants and bakeries publish donations; verified NGOs request a quantity from an existing donation; the owning business approves the entire request or rejects it; a volunteer/collector coordinates pickup; and the NGO records distribution. The application runs on localhost without Docker.

2. High-level architecture

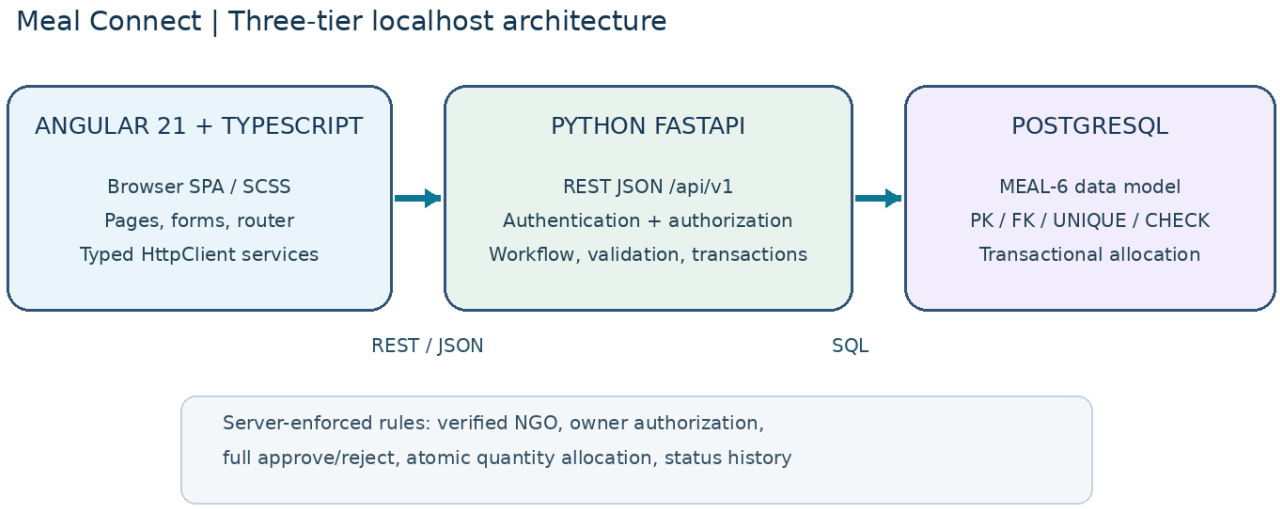


Figure 1. Angular SPA communicates with FastAPI over HTTP/JSON; only the backend accesses PostgreSQL. Diagram is logical, not a claim that every module has been implemented.

Layer	Technology	Primary responsibility
Presentation	Angular 21, TypeScript, SCSS	Role-based pages, routing, reactive forms, accessibility, typed HttpClient API calls.
Application/API	Python, FastAPI, Uvicorn	Identity, role/organization verification, input validation, workflow rules, normalized JSON responses and errors.
Persistence	PostgreSQL	MEAL-6 schema, relationships, constraints, transactions, allocation integrity and durable history.

3. Component responsibilities and boundaries

Component	What team members implement	Boundary / rule
Angular feature areas	Authentication, donation browsing, donation create/edit, NGO request, business decision, pickup, distribution, administration.	UI does not connect to PostgreSQL or make final access decisions.
Angular API services	Central HttpClient calls, typed request/response interfaces, common error display.	Base path /api/v1; exact endpoint contracts belong to MEAL-62.
FastAPI routers and schemas	Request parsing, Pydantic validation, response status codes and shared error handler.	Authentication and permission checks on protected endpoints.
FastAPI business services	Donation lifecycle, full approval/rejection, quantity allocation, pickup/distribution transitions and reports.	Never rely on client-side quantity or claimed roles.
PostgreSQL + migrations	Tables, references, uniqueness, positive quantity checks, indexes and transactional changes.	Database only accessed using backend-held credentials.

4. User journeys and information flow

1. Restaurant/bakery creates a surplus-food listing with total quantity, description and pickup deadline. FastAPI validates ownership and stores the listing.
2. Verified NGO browses eligible existing listings and submits requested_quantity. No standalone general NGO request is supported.
3. Owning restaurant/bakery approves the requested quantity in full or rejects the request; partial approval and approved_quantity fields are not part of this MVP.
4. At approval, backend checks expiry, cancellation, request state and remaining stock, then atomically updates allocated_quantity and request status in one PostgreSQL transaction.
5. Assigned collector records pickup/delivery progress; authorized NGO records final distribution; status changes are recorded as appropriate.
6. Notifications and aggregate impact reporting reflect stored confirmed actions and distribution records.

5. Data model linkage (MEAL-6)

Database entities: users; organizations; organization_memberships; organization_verifications; donations; donation_requests; pickups; distribution_records; notifications; status_history. Use the reviewed MEAL-6 DBML for exact fields, keys, enums and migration design.

Enforce at most one pickup for each donation request (UNIQUE pickups.request_id) and at most one distribution record for each pickup (UNIQUE distribution_records.pickup_id). Ensure requested quantities are positive and approved allocation never exceeds donation total_quantity. Handle simultaneous approvals safely with database locking or conditional updates inside transactions.

6. Interfaces and API boundaries

Browser → FastAPI: HTTP with JSON bodies and snake_case field names; versioned base route /api/v1. FastAPI should provide consistent 2xx responses and normalized 4xx/5xx errors. MEAL-62 is the canonical

complete endpoint, payload, permissions and error contract; do not infer schemas from this high-level guide.

FastAPI → PostgreSQL: backend connection only, with credentials stored in an ignored local .env. Avoid direct browser database connections. Cross-origin Angular development requests require FastAPI CORS configured for the actual frontend origin.

7. Local development and integration sequence

Step	Practical action
1. Source control	Use feature branches and pull requests targeting develop. Keep main as stable branch.
2. Frontend	From frontend/: npm ci then npm start; Angular normally serves http://localhost:4200.
3. Backend	Create Python virtual environment, install pinned FastAPI dependencies, run Uvicorn for local API (commonly 127.0.0.1:8000).
4. Database	Start local PostgreSQL, create project database, run MEAL-6 migrations and test connection from backend.
5. Integration	Connect Angular HttpClient services to MEAL-62 routes; verify error displays, authorization and quantity conflicts.
6. Testing	Exercise end-to-end listing → request → approval/rejection → pickup → distribution, including concurrency and expiry.

The Angular default welcome page has already been run successfully. This verifies frontend bootstrapping only; it does not demonstrate completed FastAPI or PostgreSQL integration.

8. Implementation rules and exclusions

- Server is authoritative for role membership, organization verification, ownership and workflow state; Angular guards are navigation helpers only.
- Do not support partial quantity approval. If requested stock is unavailable, return a conflict without modifying allocation.
- No native mobile apps, payment gateway, AI matching, real-time GPS, third-party delivery integrations or production deployment in this MVP.
- Do not restore legacy frontend/assets, frontend/css or frontend/js placeholder directories; use Angular CLI-generated project structure.
- Keep architecture and API documents synchronized with implemented behavior if future technical choices change.

9. Where to find implementation details

MEAL-5 v1.3: system rules and scope • MEAL-6: database schema and constraints • MEAL-62: complete API contracts • MEAL-63: backend setup • MEAL-163: tools and versions • MEAL-197 / MEAL-209: project guidance and risk decisions.